

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The **Nykaa E-Commerce Database Management System** is designed to efficiently manage customers, products, categories, orders, payments, deliveries, and administrators. Functional dependencies help identify the relationships among attributes in each table, while redundancy analysis and anomaly detection ensure data consistency, minimize duplication, and improve database performance. A properly normalized database supports efficient order processing and maintains data integrity.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → **Customer_Name, Email, Phone_Number, Address**

Redundancy Analysis

Customer details are stored only once in the Customer table. Orders reference the customer using **Customer_ID**, preventing duplicate customer information.

Data Anomalies

- **Insert Anomaly:** A customer can register without placing an order.
 - **Update Anomaly:** Updating a customer's email or phone number requires changes only in the Customer table.
 - **Delete Anomaly:** Deleting an order does not remove customer details.
-

2. SELLER TABLE

Functional Dependency

Seller_ID → **Seller_Name, Email, Phone_Number, Address**

Redundancy Analysis

Seller information is maintained in a separate table. Products store only the **Seller_ID**, avoiding repeated seller details.

Data Anomalies

- **Insert Anomaly:** A seller can be added without listing products.
 - **Update Anomaly:** Seller information is updated only once.
 - **Delete Anomaly:** Removing a product does not delete seller information.
-

3. PRODUCT TABLE

Functional Dependency

Product_ID → **Product_Name, Brand, Price, Stock, Category, Seller_ID**

Redundancy Analysis

Product details are stored only once in the Product table. Orders reference products using **Product_ID**, eliminating duplicate product records.

Data Anomalies

- **Insert Anomaly:** Products can be added before any customer purchases them.
 - **Update Anomaly:** Product price or stock is updated only once.
 - **Delete Anomaly:** Deleting an order does not remove product details.
-

4. ORDER TABLE

Functional Dependency

Order_ID → **Customer_ID, Order_Date, Total_Amount, Order_Status**

Redundancy Analysis

The Order table stores only order-related information. Customer details are accessed using **Customer_ID**.

Data Anomalies

- **Insert Anomaly:** Orders can be created only for registered customers.
 - **Update Anomaly:** Order status is updated only in the Order table.
 - **Delete Anomaly:** Deleting an order does not remove customer information.
-

5. PAYMENT TABLE

Functional Dependency

Payment_ID → **Order_ID**, **Payment_Method**, **Payment_Status**, **Payment_Date**

Redundancy Analysis

Payment information is stored separately from orders, preventing duplicate payment records.

Data Anomalies

- **Insert Anomaly:** Payment details can be added after an order is created.
 - **Update Anomaly:** Payment status is updated only once.
 - **Delete Anomaly:** Deleting payment details does not remove the corresponding order.
-

6. INVENTORY TABLE

Functional Dependency

Inventory_ID → **Product_ID**, **Stock_Quantity**, **Warehouse_Location**

Redundancy Analysis

Inventory details are stored separately from product details. Products reference inventory using **Product_ID**, reducing data duplication.

Data Anomalies

- **Insert Anomaly:** Inventory records can be created before products are sold.
- **Update Anomaly:** Stock quantity changes are made only once.

- **Delete Anomaly:** Deleting an inventory record does not remove product information.
-

7. REVIEW TABLE

Functional Dependency

Review_ID → **Customer_ID**, **Product_ID**, **Rating**, **Review_Text**, **Review_Date**

Redundancy Analysis

Customer reviews are stored in a separate table. Only **Customer_ID** and **Product_ID** are referenced, avoiding repeated customer and product information.

Data Anomalies

- **Insert Anomaly:** A review can be added only for an existing product and customer.
 - **Update Anomaly:** Editing a review affects only that review record.
 - **Delete Anomaly:** Deleting a review does not remove customer or product details.
-

Conclusion

The **Nykaa E-Commerce Database** is designed using proper functional dependencies and normalized tables. Each entity stores only its own relevant information, reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains referential integrity, and ensures efficient management of customers, sellers, products, orders, payments, inventory, and reviews.